Failure Memory in Grid-World Agents

Ishaan Verma
Undergraduate student, University of Bristol
ORCID: https://orcid.org/0009-0006-2038-3274
Independent research preprint

July 2026

Keywords: failure memory, grid-world navigation, embodied agents, reactive navigation, reproducible evaluation

Abstract

An agent that acts over many steps can waste effort by attempting the same blocked move more than once. This study measures whether persistent within-run memory of blocked state-action pairs reduces that waste in a simple grid world, and whether the reduction changes task outcomes. Two agents share an identical deterministic policy that ranks moves by Manhattan-distance reduction and breaks ties with a seeded action permutation. They differ in one respect. The baseline agent excludes a blocked pair for the next decision only, which gives immediate reactive recovery. The memory agent excludes a blocked pair for the rest of the run. The comparison uses 10 fixed mazes and 5 seeds, giving 100 paired runs. Persistent memory removed repeated blocked actions, reducing the mean from 35.12 to 0.00 per run, and cut mean blocked actions from 37.00 to 2.04. Success rate and mean total steps did not change: both agents reached the goal in 56 percent of runs and used 65.5 steps on average. The two agents behaved identically on the five mazes that both solve. They differed only on the five mazes where the baseline became trapped. On those mazes both agents still failed, because the greedy policy fell into oscillations between valid, unblocked cells, which blocked-action memory cannot detect. Persistent failure memory therefore reduced one class of repeated error without improving overall navigation. All code, data, and figures are released, and the full run reproduces to an identical fingerprint.

1. Introduction

Embodied agents choose actions in sequence, and a single wrong move can be repeated many times if the agent keeps no record of it. In a grid world with walls, an agent that heads toward its goal will sometimes select a move into a wall. If the agent forgets that outcome immediately, it can select the same blocked move again on a later visit to the same cell. Storing failed state-action pairs is a direct way to prevent that specific error, and the idea has a long history in reactive navigation, where an automaton that remembers where it has already met an obstacle can still reach its goal

Recent work on learned agents adds memory in richer forms, including external read-write memory (Graves et al., 2016), episodic buffers of past experience (Blundell et al., 2016; Pritzel et al., 2017), and spatial or topological memory for navigation (Parisotto and Salakhutdinov, 2018; Savinov et al., 2018). These systems couple memory with learning and perception, which makes it hard to attribute any single result to the memory component alone. This study takes the opposite approach and isolates one minimal mechanism. It asks whether a plain set of blocked state-action pairs, remembered for the duration of a run, reduces repeated navigation errors beyond what immediate reactive feedback already provides.

The research question is: does persistent within-run memory of blocked state-action pairs reduce repeated navigation errors beyond immediate reactive feedback in simple grid-world agents? To answer it, the baseline is stronger than a memoryless agent that loops on a wall until timeout. The baseline has transient recovery: it avoids a blocked move for the next decision only, which removes the easy target and isolates the effect of persistence. The memory agent keeps the same exclusion for the rest of the run. The single controlled difference is the horizon over which a blocked pair is remembered.

The paper makes four contributions. First, it provides a controlled comparison between transient one-decision feedback and persistent within-run failure memory under an otherwise identical policy. Second, it provides a reproducible paired grid-world benchmark with released code and data. Third, it reports evidence that failure memory can remove repeated blocked actions without improving task completion or path length. Fourth, it analyses the failure mode that remains after the repeated blocks are removed, namely oscillation between valid states, and explains why blocked-action memory does not address it. The fourth point matters for anyone who expects a memory of failures to improve navigation on its own, because it shows one class of failure that this kind of memory leaves untouched.

2. Related work

2.1 Memory in autonomous and learned agents

Explicit memory has been used to extend what neural agents can store and reuse. A network coupled to a differentiable external memory can hold information over long timescales and manipulate it like a data structure (Graves et al., 2016). Episodic control methods keep a nonparametric record of past states and returns and use it to guide action selection, which can speed up early learning (Blundell et al., 2016; Pritzel et al., 2017). These methods store rich content, such as state embeddings and value estimates, and update it through learning. The memory studied here is far simpler. It stores only discrete (state, action) pairs that produced a blocked outcome, it holds no values, and it is not learned.

2.2 Memory for navigation and planning

Navigation agents have used memory structured for space and for past observations. A structured spatial memory can store information about the environment across an episode and improve navigation over an agent limited to the last few frames (Parisotto and Salakhutdinov, 2018). A semi-parametric topological memory records visited places as graph nodes and supports goal-directed navigation in new environments (Savinov et al., 2018). Auxiliary objectives related to memory, such as predicting loop closure, improve navigation performance in complex mazes (Mirowski et al., 2017). These systems remember places and observations. The present study remembers failures rather than places, which is a narrower signal, and it stores that signal without any function

approximator.

2.3 Reactive navigation, grid-world evaluation, and reproducibility

Reactive navigation has a well-documented weakness. Local methods such as potential fields are prone to local minima and to oscillatory motion, for example in narrow passages (Koren and Borenstein, 1991). This weakness is central to the present results, because the residual failures observed here are oscillations of exactly that kind. Grid worlds remain a standard controlled testbed for sequential decision-making (Sutton and Barto, 2018), and minimalistic grid-world suites are widely used to build small, customizable agent tasks (Chevalier-Boisvert et al., 2023). Finally, work on reproducibility in agent research argues that variance and inconsistent reporting make results hard to interpret and that fixed conditions and released code are needed for credible comparison (Henderson et al., 2018). This study follows that guidance with a deterministic design, fixed seeds, released code and data, and a verifiable fingerprint of the raw results.

3. Methodology

3.1 Environment

The environment is a bounded rectangular grid indexed by row and column, with row indices increasing downward. Each maze fixes the grid size, a start cell, a goal cell, and a set of wall cells. The action set is up, down, left, and right, each a unit move. A move is blocked when its target cell lies outside the grid or is a wall, and a blocked move leaves the agent in place. The environment returns the new position, whether the move was blocked, and whether the goal was reached. It holds no state beyond the current position and applies the same rules to both agents.

The step limit for a maze is four times its traversable-cell count, where traversable cells are the grid cells that are not walls. The limit is computed before the run and is identical for both agents on a maze. A breadth-first search confirms, before any run, that a path from start to goal exists and that its shortest length is below the step limit. The agents never receive the breadth-first search result, so it cannot influence behaviour.

3.2 Shared policy

Both agents use the same deterministic policy, and action selection consumes no random values. At run start, one action-priority permutation of the four actions is derived from the seed. At each decision, the four actions are ranked by the reduction in Manhattan distance to the goal, computed from the geometric target cell without any knowledge of walls, so distance-increasing moves are still ranked. Ties in reduction are broken only by the permutation. The agent selects the highest-ranked available action. Because nothing is drawn from a random number generator during selection, the two agents cannot diverge through random noise, and any difference in behaviour comes from the memory alone.

3.3 Agents and the single difference

The two agents share all of the above and differ only in how long a blocked pair is excluded. The baseline agent, agent A, uses transient exclusion. When a (state, action) pair is blocked, it excludes that pair for the immediately following decision only, then clears the exclusion. If it returns to the same cell later, it may attempt the same move again. Agent A therefore has immediate reactive recovery but no lasting memory within the run. The memory agent, agent B, uses persistent

exclusion. It stores every blocked pair for the rest of the run and keeps excluding it on later visits to the same cell, and it clears the store only at the start of the next run. If exclusions would remove every action at a cell, the agent ignores exclusions for that one decision and takes the highest-ranked action, which prevents a deadlock. Figure 1 shows the shared pipeline and the single difference.

One decision is one action attempt and one step. Blocked attempts count toward total steps and toward the step limit. After a block the agent stays in place and selects again, which is the next decision at the same cell.

3.4 Metrics

All metrics are computed from the external action log, not from any agent's memory, so both agents are scored by one rule. Success is 1 if the goal is reached within the step limit. Total steps counts every attempt, including blocked ones. Blocked actions counts attempts that returned a blocked result. A repeated failed action is an attempt of a (state, action) pair that already produced a blocked outcome earlier in the same run, with membership checked before the current attempt is recorded.

3.5 Design and reproducibility

The main design uses 10 fixed mazes and 5 seeds for 2 agents, giving 100 runs. Each (maze, seed) pair is a matched pair: agent A and agent B run on the same maze, seed, permutation, and step limit, sharing a pair identifier. The five seeds produce five distinct action permutations. Raw results are written to a comma-separated file with one row per run, and full per-step traces record the candidate actions, active exclusions, chosen action, outcome, and memory update. The experiment is deterministic. Two runs produced the identical raw-row fingerprint a40f7100e1406a6a0b1d06a2826752bafc7a6570b195c850c752d1cdc0efd93c. The code and data are released so the run can be repeated with a single command.

4. Results

4.1 Aggregate results

Table 1 reports the aggregate results over 50 runs per agent. Persistent memory reduced the mean repeated failed actions from 35.12 to 0.00 and the mean blocked actions from 37.00 to 2.04. Agent B recorded no repeated failed action on any of its 50 runs, while agent A recorded 1756 repeated failed actions in total. Success rate and mean total steps were the same for both agents, at 0.56 and 65.50. Figures 2 and 3 show the differences in repeated failed actions and blocked actions, and Figure 4 shows that mean total steps is unchanged.

Table 1: Aggregate results across 50 runs per agent.

Metric	Agent A (transient)	Agent B (persistent)
Success rate	0.56	0.56
Mean total steps	65.50	65.50
Mean blocked actions	37.00	2.04
Mean repeated failed actions	35.12	0.00

4.2 Where the agents differ

The two agents behaved identically on the five mazes that both solve, which are maze_01, maze_02, maze_04, maze_06, and maze_07. On maze_02, maze_04, and maze_07 both agents hit walls, so their blocked-action counts are nonzero and equal, yet neither agent repeated a blocked action. On these mazes agent A's single-decision recovery was already enough, and persistence changed nothing.

The agents differed only on the five mazes where agent A became trapped, which are maze_03, maze_05, maze_08, maze_09, and maze_10. On these mazes agent A accumulated many repeated blocked actions while agent B removed them. Figure 5 shows the per-maze repeated failed actions and makes clear that the whole effect comes from these five mazes. The size of agent A's effect depends on the seed, because the permutation changes which moves agent A cycles through. On maze_08, for example, agent A repeated 45 blocked actions under three seeds and 135 under the other two, while agent B repeated none under any seed.

4.3 The null result on task outcomes

Success rate and total steps did not improve with memory. Both agents solved the same 56 percent of runs and used the same number of steps. On the five hard mazes, both agents failed, and maze_10 was solved by both agents under three of five seeds and failed by both under the other two. Persistent memory reduced wasted actions within runs that failed for a different reason, which the next section examines.

5. Discussion

The result is a partial success, and the two halves should be stated separately.

Agent B solved the narrow problem it was designed to solve. It remembered blocked state-action pairs, it avoided repeating them, and it reduced blocked and repeated failed actions to near zero. This is a clean, measured effect, and it is exactly what a persistent record of failures should produce.

Agent B did not solve the harder problem of reaching the goal. On the five mazes where the two agents differ, both still failed. Inspection of the traces explains why. After agent B removes the blocked moves at a cell, the greedy policy settles into a cycle between two valid cells. On maze_08 under seed 3, agent B ends in a cycle between cells (0, 6) and (0, 7). At (0, 7) the move toward the goal is walled and is excluded, so the agent steps left to (0, 6); at (0, 6) the greedy move toward the goal is right, which returns it to (0, 7). Both moves in this cycle are valid and are never blocked, so the failure memory never records them and cannot break the loop. Figure 6 shows this cycle. The same pattern appears on maze_10 under seed 3, where agent B cycles between (3, 8) and (4, 8) using valid up and down moves. This kind of oscillation in local reactive navigation is a known limitation of policies that follow a local gradient (Koren and Borenstein, 1991), and blocked-action memory does not change it because the moves involved are never blocked.

A compact way to state the interpretation is that failure memory improved local error efficiency without improving global task performance in this environment. The agent wasted far fewer actions on moves it had already found to be blocked, which is a real gain in efficiency of a specific kind, yet it did not reach more goals or take shorter paths. This interpretation is supported by the present environment and policy and should not be read as a general law. A different base policy, or a memory that also records visited cells or short trajectories, could change the outcome, and testing that is future work rather than a claim of this paper.

The comparison also shows where memory added nothing. On the five mazes that both agents solve, agent A's transient recovery already avoided repeats, so agent B matched it exactly. Persistent memory helped only when the baseline would otherwise revisit and repeat a blocked move, which happened on the harder mazes. A memory of failures is therefore useful in proportion to how often the base policy would repeat a failure, and it is silent when the base policy does not.

6. Limitations and future work

The study has clear limits, and they bound what the results support. The environment is a small deterministic grid world with no partial observability, no sensor noise, and no physical robot. The base policy is a fixed Manhattan-distance rule with no learned component and no memory of visited cells. The agents use no language model and no planner. The maze set is small and hand-designed. The five seeds vary only the tie-breaking permutation and do not sample a population of environments, so they support description rather than statistical inference, and no significance test was run or is claimed (Henderson et al., 2018). The memory stores only blocked state-action pairs, and it cannot recognise cycles made entirely of valid moves, which is the failure mode that dominated the harder mazes.

Future work can address these limits in specific ways. Adding detection of cycles among valid states, for example by remembering recently visited cells, would target the oscillation directly. Comparing state-action memory against short trajectory memory would show which representation prevents more failures. Making the memory revisable would allow use in environments that change during a run. Replacing the greedy rule with a stronger planner would test whether failure memory still helps once local minima are handled. Integrating the environment with a robotics simulator such as a ROS 2 stack would test the mechanism under continuous dynamics. Evaluating memory shared across several agents would show whether one agent's recorded failures help another.

7. Conclusion

Persistent within-run memory of blocked state-action pairs removed repeated blocked actions in a controlled grid-world comparison, reducing the mean from 35.12 to 0.00 per run and cutting mean blocked actions from 37.00 to 2.04. The same memory did not raise the success rate or shorten paths, which stayed at 0.56 and 65.50 for both agents. The remaining failures came from oscillation between valid cells, a failure mode that blocked-action memory does not detect. The narrow finding is that remembering blocked state-action pairs reduces collision-like repeated errors, and that this reduction did not translate into higher task success or shorter runs under the tested policy.

8. Code and data availability

The implementation, fixed maze definitions, experiment traces, raw results, analysis scripts, tests, and reproduction instructions are available at https://github.com/PliableRigidity/failure-memory-gridworld.

The full experiment can be reproduced with:

```
python -m experiments.run_full
```

9. Declaration of generative AI assistance

Generative AI tools were used for software-development assistance, literature discovery, and language editing. The author designed and verified the experiments, reviewed the sources, interpreted

the results, and takes responsibility for the final work.

Appendix A. Figures

The figures referenced in the text are collected in this appendix in reference order.

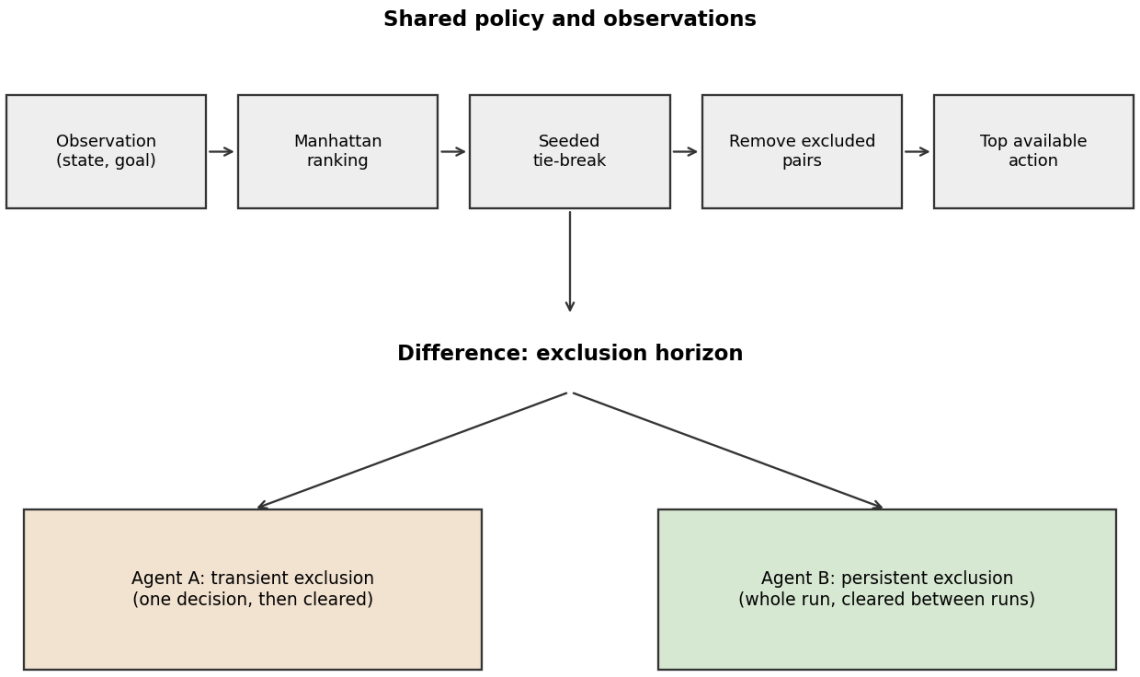

Figure 1. Shared policy and the single difference between the two agents. Both agents share the observation, Manhattan-distance ranking, seeded tie-breaking, removal of excluded state-action pairs, and selection of the top available action. They differ only in the exclusion horizon: Agent A clears an exclusion after one decision, and Agent B keeps it for the whole run.

Figure 2. Mean repeated failed actions per run for the transient and persistent-memory agents (35.12 and 0.00).

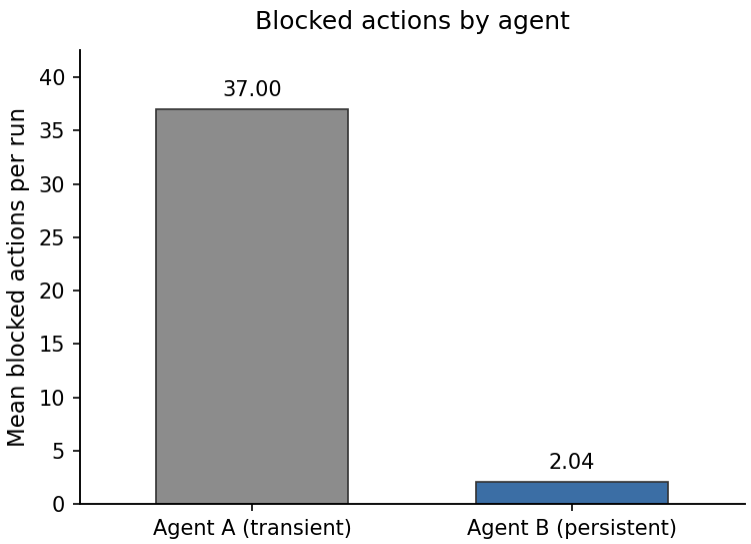

Figure 3. Mean blocked actions per run for the transient and persistent-memory agents (37.00 and 2.04).

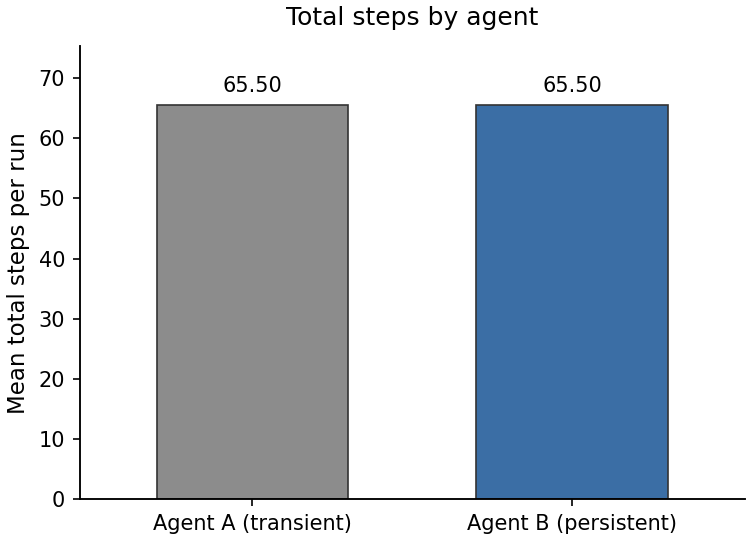

Figure 4. Mean total steps per run for the transient and persistent-memory agents (65.50 for both).

Figure 5. Mean repeated failed actions per maze for the transient and persistent-memory agents. The effect comes from maze_03, 05, 08, 09, and 10.

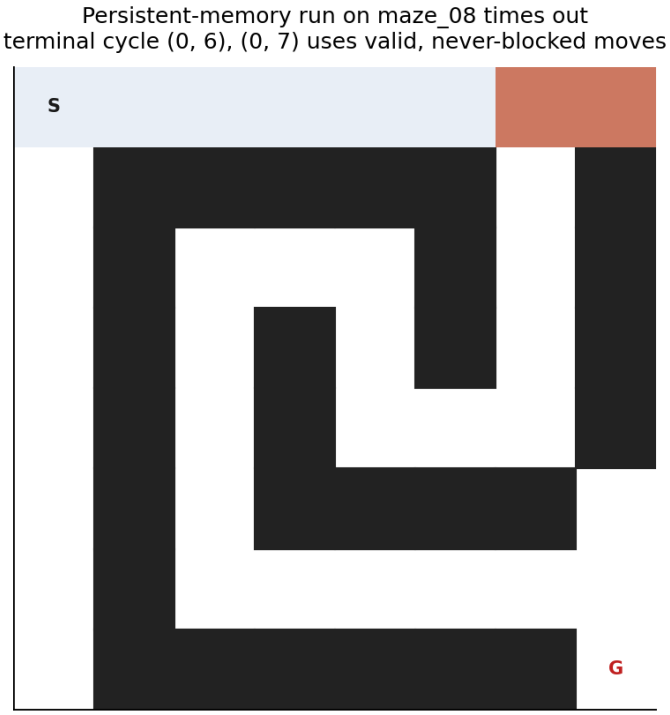

Figure 6. A persistent-memory run on maze_08 (seed 3) that times out. The agent settles into a cycle between cells (0, 6) and (0, 7) using valid moves that are never blocked, so failure memory cannot break the loop.

References

Blundell, C., Uria, B., Pritzel, A., Li, Y., Ruderman, A., Leibo, J. Z., Rae, J., Wierstra, D., and Hassabis, D. (2016). Model-Free Episodic Control. arXiv preprint arXiv:1606.04460. Preprint, not peer reviewed.

Chevalier-Boisvert, M., Dai, B., Towers, M., de Lazcano, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. (2023). Minigrid and Miniworld: Modular and Customizable Reinforcement Learning Environments for Goal-Oriented Tasks. Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track. arXiv:2306.13831.

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwinska, A., Colmenarejo, S. G., Grefenstette, E., Ramalho, T., Agapiou, J. P., Puigdomenech Badia, A., Hermann, K. M., Zwols, Y., Ostrovski, G., Cain, A., King, H., Summerfield, C., Blunsom, P., Kavukcuoglu, K., and Hassabis, D. (2016). Hybrid computing using a neural network with dynamic external memory. Nature, 538(7626), 471-476. doi:10.1038/nature20101.

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2018). Deep Reinforcement Learning That Matters. Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18). doi:10.1609/aaai.v32i1.11694. arXiv:1709.06560.

Koren, Y., and Borenstein, J. (1991). Potential field methods and their inherent limitations for mobile robot navigation. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 1398-1404. doi:10.1109/ROBOT.1991.131810.

Lumelsky, V. J., and Stepanov, A. A. (1987). Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape. Algorithmica, 2(1), 403-430. doi:10.1007/BF01840369.

Mirowski, P., Pascanu, R., Viola, F., Soyer, H., Ballard, A. J., Banino, A., Denil, M., Goroshin, R., Sifre, L., Kavukcuoglu, K., Kumaran, D., and Hadsell, R. (2017). Learning to Navigate in Complex Environments. International Conference on Learning Representations (ICLR). arXiv:1611.03673.

Parisotto, E., and Salakhutdinov, R. (2018). Neural Map: Structured Memory for Deep Reinforcement Learning. International Conference on Learning Representations (ICLR). arXiv:1702.08360.

Pritzel, A., Uria, B., Srinivasan, S., Puigdomenech Badia, A., Vinyals, O., Hassabis, D., Wierstra, D., and Blundell, C. (2017). Neural Episodic Control. Proceedings of the 34th International Conference on Machine Learning (ICML), PMLR 70, 2827-2836.

Savinov, N., Dosovitskiy, A., and Koltun, V. (2018). Semi-parametric Topological Memory for Navigation. International Conference on Learning Representations (ICLR). arXiv:1803.00653.

Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction, second edition. MIT Press. ISBN 9780262039246.